Mobile Wallet Fraud Detection Using Machine Learning

Rowida Gamal Hassan

Problem Statement

The Problem

- Mobile wallets make financial transactions fast and convenient.
- Fraudulent transactions can cause financial losses.
- Fraud detection is challenging because fraudulent transactions are extremely rare compared with legitimate ones.

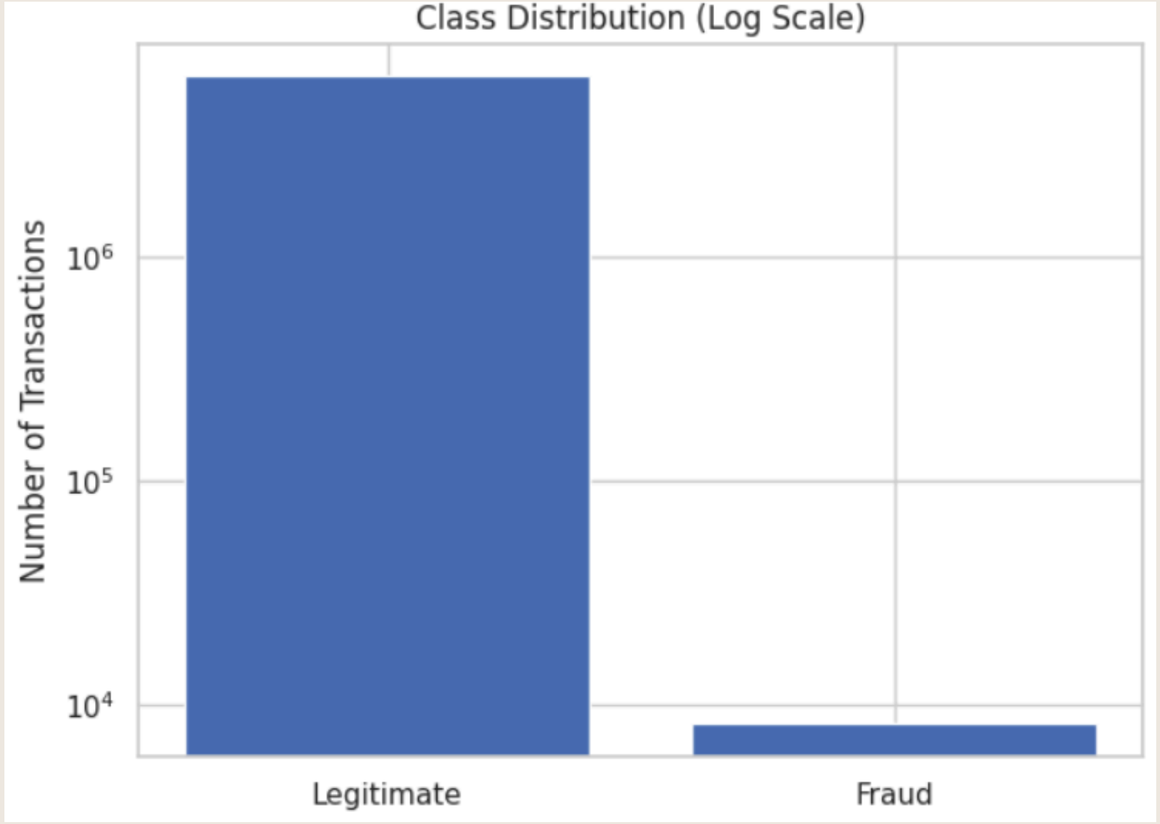
Project Objective

- Build a machine learning system that identifies potentially fraudulent transactions while minimizing missed fraud cases and unnecessary false alarms.

Dataset Overview

PaySim Mobile Money Dataset

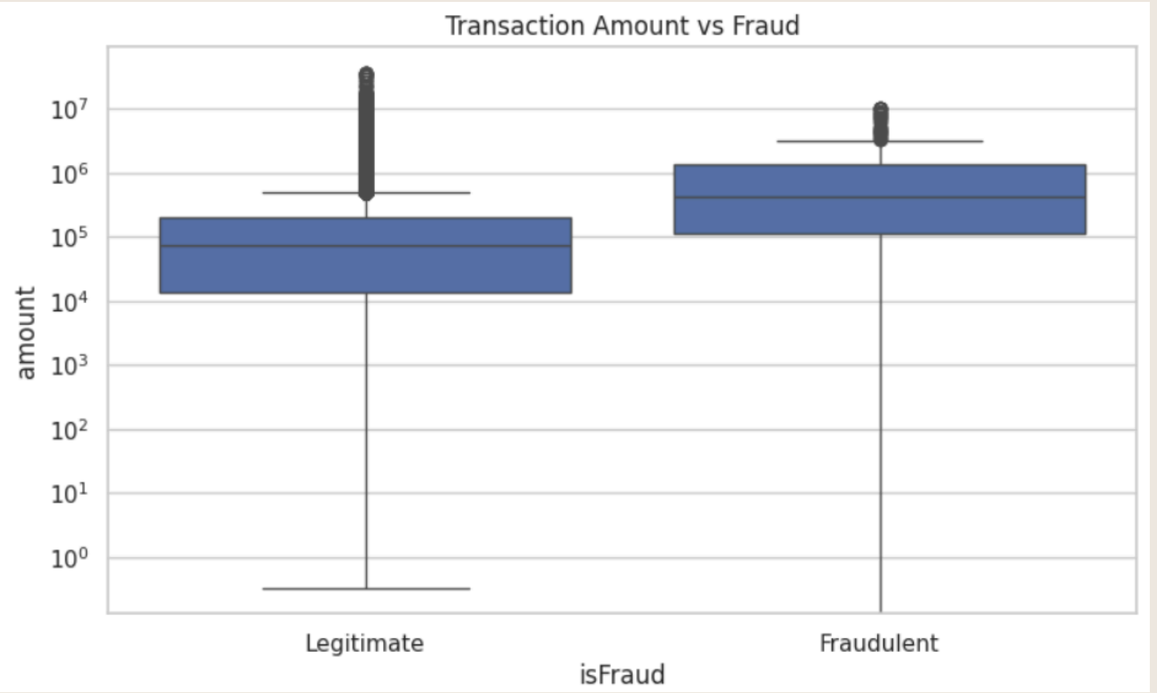
- Total transactions: 6,362,620
- Total features: 11
- Legitimate transactions: 6,354,407
- Fraudulent transactions: 8,213
- Fraud percentage: 0.129%
- Dataset Source: PaySim on Kaggle.

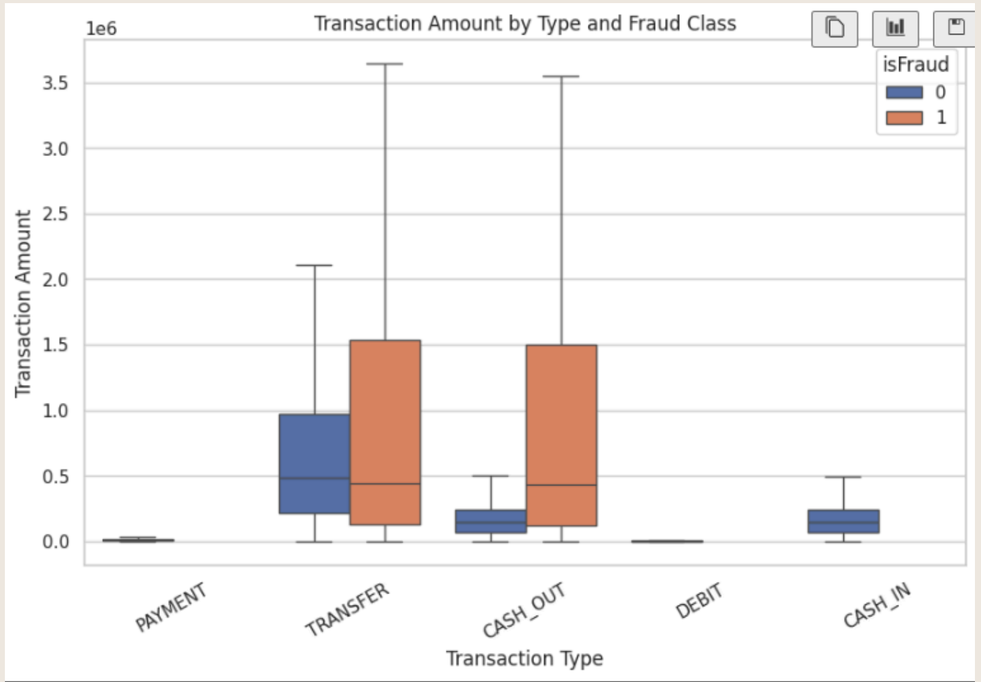


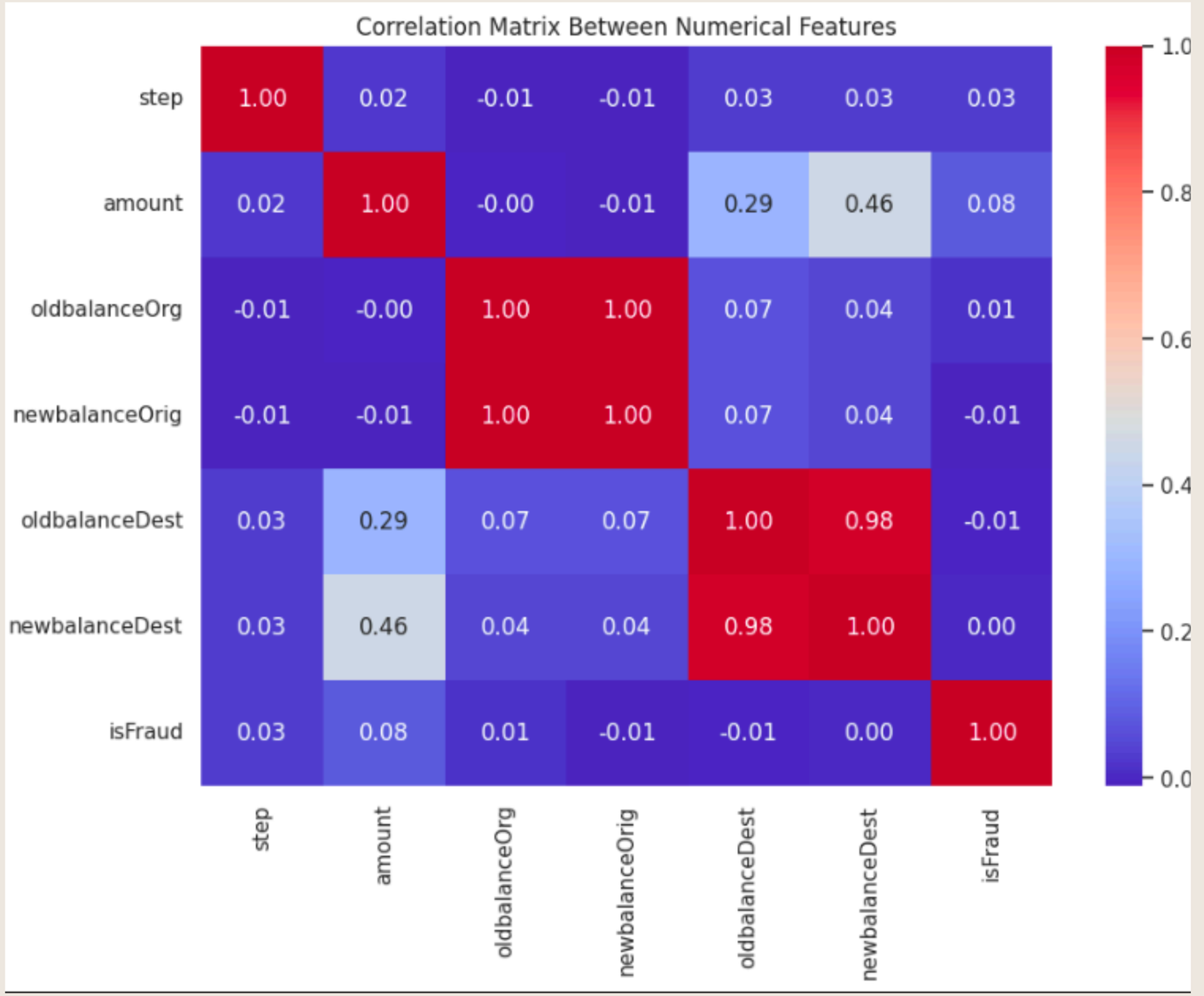
Exploratory Data Analysis

Key EDA Insights

- The dataset has severe class imbalance.
- Fraudulent transactions represent only approximately 0.13% of all transactions.
- Investigated transaction types, transaction amounts, and account balance patterns.
- In this dataset, fraud was associated with TRANSFER and CASH_OUT transaction types.


Transaction Amount vs Fraud
amount
10^7
10^6
10^5
10^4
10^3
10^2
10^1
10^0
Legitimate
Fraudulent
isFraud


Transaction Amount by Type and Fraud Class
1e6
isFraud
0
1
Transaction Amount
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
PAYMENT
TRANSFER
CASH_OUT
DEBIT
CASH_IN
Transaction Type


Correlation Matrix Between Numerical Features
step 1.00 0.02 -0.01 -0.01 0.03 0.03 0.03
amount 0.02 1.00 -0.00 -0.01 0.29 0.46 0.08
oldbalanceOrg -0.01 -0.00 1.00 1.00 0.07 0.04 0.01
newbalanceOrig -0.01 -0.01 1.00 1.00 0.07 0.04 -0.01
oldbalanceDest 0.03 0.29 0.07 0.07 1.00 0.98 -0.01
newbalanceDest 0.03 0.46 0.04 0.04 0.98 1.00 0.00
isFraud 0.03 0.08 0.01 -0.01 -0.01 0.00 1.00
step
amount
oldbalanceOrg
newbalanceOrig
oldbalanceDest
newbalanceDest
isFraud
1.0
0.8
0.6
0.4
0.2
0.0

Data Preprocessing

Preparing the Data

- Selected the target variable: isFraud.
- Selected five original input features:
  - step
  - type
  - amount
  - oldbalanceOrg
  - oldbalanceDest
- Split data into training, validation, and test sets using stratified splitting.
- Used a Scikit-learn Pipeline and ColumnTransformer.
- Applied median imputation and RobustScaler to numerical features.
- Applied most-frequent imputation and OneHotEncoder to categorical features.

Important: Excluded post-transaction balance features to prevent data leakage.

Feature Engineering

Creating More Informative Features

Generated additional features, including:

- hour_of_day
- day_number
- log_amount
- amount_to_origin_balance
- amount_to_destination_balance
- origin_balance_insufficient

These features help the model capture transaction timing, amount scale, and account balance relationships.

Models and Experimentation

Model Development

- Established a Dummy Classifier baseline.
- Trained and compared multiple classification algorithms.
- Logistic Regression , Decision Tree , Random Forest
- Used cross-validation to evaluate model performance.
- Applied hyperparameter tuning to improve the selected model.
- Used PR-AUC as an important evaluation metric because of the severe class imbalance.

Best Model: Random Forest

Best hyperparameters:

- n_estimators = 150
- max_depth = 20
- min_samples_leaf = 2
- class_weight = balanced

Best cross-validation PR-AUC: 0.98834.

Threshold Optimization

Business-Based Threshold Selection

- The default classification threshold is 0.50.
- A lower threshold can help detect more fraudulent transactions.
- Selected a threshold of approximately 0.10 using validation data.
- Considered the cost of missed fraud and false alarms.

Assumed costs:

- False Negative: 1,000
- False Positive: 10

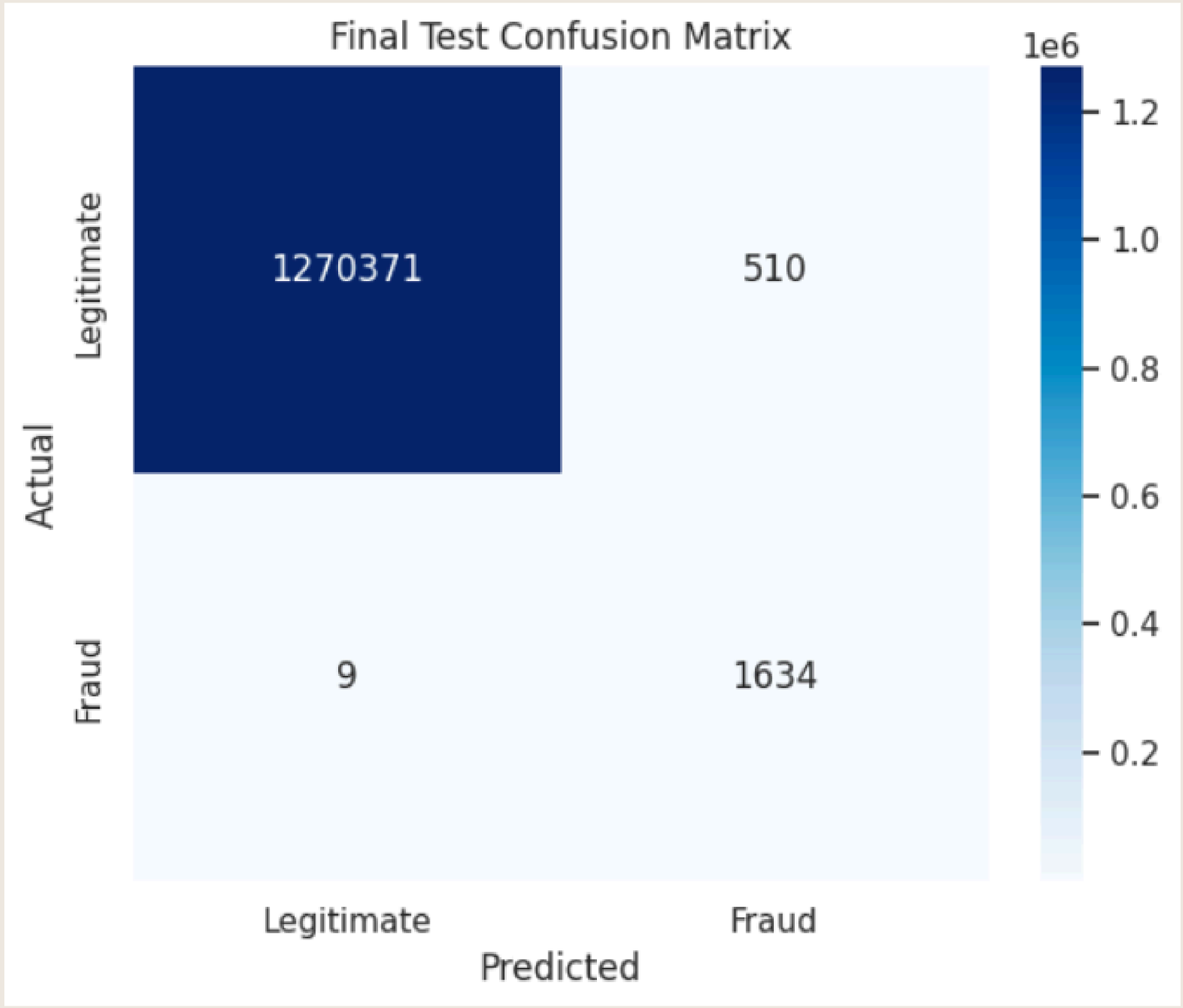
Minimum validation business cost: 6,050.

Final Test Results

METRIC	RESULT
Accuracy	0.999592
Precision	0.762127
Recall	0.994522
F1-Score	0.862952
PR-AUC	0.993402
ROC-AUC	0.99903

Confusion Matrix Results

- True Positives: 1,634
- True Negatives: 1,270,371
- False Positives: 510
- False Negatives: 9



Application and Deployment

Interactive Streamlit Application

The application provides:

1. Single transaction prediction.
2. CSV upload for batch predictions.
3. Fraud probability and classification.
4. Evaluation metrics when actual labels are provided.
5. Downloadable prediction results.

Live Demo:

https://nextgen-fraud-detection.streamlit.app/

GitHub:

https://github.com/rowida-gamal4/NextGen_FinalProject

Recommendations, Limitations and Conclusion

Business Recommendations

- Use risk-based transaction screening.
- Prioritize transactions with high predicted fraud probabilities.
- Review flagged transactions before taking action.
- Monitor false positives and missed fraud cases.
- Reevaluate the threshold using real operational costs.

Limitations

- PaySim is a synthetic dataset.
- Real-world fraud patterns may differ.
- The assumed business costs require validation.
- Real-world deployment requires further testing and monitoring.

Conclusion

The Random Forest model achieved 99.34% PR-AUC and 99.45% recall on an independent test set, and was successfully deployed through an interactive Streamlit application.

THANK YOU